OFFLINE-FIRST BUSINESS APP STARTER KIT

Developer User Manual

Metadata-Driven Framework

Adding · Deploying · Maintaining Client Systems

Version 1.0 · July 2026

Metadata-Driven Framework: Adding, Maintaining & Managing Client Systems

> ***Who is this for?*** *You — the developer who built this app and needs to add new client systems, maintain existing ones, and keep everything organized in one codebase.*
>
> ***What will you learn?*** *Exactly what to do, step by step, in plain language — no assumptions, no jargon overload.*

■ Table of Contents

---

1. Big Picture — How This App Works (5-Minute Summary)

Think of this app like a **LEGO set for business software**:

```
ONE CODEBASE

packages/ apps/ Infrastructure

(LEGO bricks) web, mobile (DB, Sync, Auth, Audit)
```

```
  |
  +-------- Client A (Cooperative, tenantId=A)
  +-------- Client B (Clinic, tenantId=B)
  +-------- Client C (Water Station, tenantId=C)
```

The key insight: You never write three separate apps. You write one app with **plug-in modules**. Each client gets their own tenant ID. The same database tables hold all clients' data — but each client can only see their own.

The 5 Core Concepts

Concept	Plain Language Meaning
Entity Package	A self-contained business module (`packages/entity-clinic`). Drop it in and it works.
Entity Registry	The app's "phone book" that knows about all modules. Modules self-register on import.
Tenant ID	A unique ID per client. All data is tagged with this. Client A can never see Client B's data.
Metadata Store	Per-client settings stored as JSON (interest rates, custom fields, workflows, themes).
Feature Flags	On/off switches for each module per client. Clinic ON for Hospital Client, OFF for Cooperative.

---

2. The Golden Rule — One Codebase, Many Clients

- ■ ***Do this:*** *Add new modules as* `packages/entity-[name]`*. Enable them per client via feature flags and tenant metadata.*

- ■ ***Don't do this:*** *Copy the whole project folder for each client, maintain separate branches per client, or hardcode client-specific logic in the core app.*

How Client Isolation Works

```
Browser (Client A — Cooperative)
|
+-- reads tenantId = "cooperative-a-id" from localStorage
|
+--> memberRepo.findMany({...})
|
v [Tenant Middleware — automatic]
SELECT * FROM members WHERE tenantId = 'cooperative-a-id'
|
v
Returns ONLY Cooperative A's members ■
```

Client B (Clinic) uses `tenantId = "clinic-b-id"` — zero data overlap.

---

3. Your Toolkit — What's Already Built

Packages You Can Reuse

Package	What It Does	When to Use
`@repo/core`	Interfaces, types, error classes, base entity shape	Always
`@repo/db-dexie`	IndexedDB (offline) database adapter	Web apps (already wired)
`@repo/multi-tenant`	Tenant isolation, metadata store	Always included automatically

Package	What It Does	When to Use
`@repo/audit-trail`	Records every change with who/when	Always included automatically
`@repo/feature-flags`	Toggle modules on/off per tenant	For per-client module control
`@repo/sync-engine`	Syncs local data to server when online	Included automatically
`@repo/ui-core`	Pre-built UI (Card, Button, Input, Badge, Modal)	Use these in route pages
`@repo/codegen`	CLI scaffold for new entity packages	`pnpm codegen entity [Name]`

Apps Already Available

- `apps/web` — React PWA (main web app — what you'll work in most)
- `apps/api` — Hono sync backend
- `apps/mobile` — Expo (iOS/Android)
- `apps/desktop` — Tauri (Windows/Mac/Linux)

---

4. Adding a Brand New Client System (Step-by-Step)

> ***Scenario:*** *A hospital client wants a Clinic Management System. You need to add it without breaking anything for existing clients.*

This is the **complete, official process**. Follow these steps every time.

STEP 1: Plan Your Entities

Before writing any code, decide what **data tables** (entities) you need. One entity = one database table.

```
Clinic Management System needs:

clinic_patients — Who are the patients?

clinic_doctors — Who are the doctors?

clinic_appointments — When do they meet?

clinic_consultation_records — What happened during the visit?

clinic_billing — How much does the patient owe?
```

Pro tip: Keep entities focused. One table = one concept.

STEP 2: Create the Entity Package

Every new system lives in `packages/entity-[name]/`.

Run the scaffold command:

```
pnpm codegen entity Clinic

# Creates: packages/entity-clinic/
```

Or create manually:

```
packages/

■■■ entity-clinic/

■■■ package.json

■■■ tsconfig.json

■■■ src/

■■■ clinic.schema.ts <- Data types + Zod validation

■■■ clinic.entity.ts <- Entity registry definitions

■■■ clinic.service.ts <- Business logic (no DB, no UI)

■■■ clinic.policies.ts <- Who can do what (RBAC)

■■■ index.ts <- Barrel export (public API)
```

package.json

```
{
"name": "@repo/entity-clinic",
"version": "0.1.0",
"private": true,
"type": "module",
"main": "./src/index.ts",
"types": "./src/index.ts",
"exports": { ".": "./src/index.ts" },
"dependencies": {
"@repo/core": "workspace:*",
"zod": "^3.23.0"
}
}
```

tsconfig.json

```
{
"extends": "../../tsconfig.base.json",
"compilerOptions": { "outDir": "dist", "rootDir": "src", "composite": true, "jsx":
"react-jsx" },
"include": ["src"],
"references": [{ "path": "../core" }]
}
```

`clinic.schema.ts` — *Define your data shape*

```
import { z } from 'zod'

import { emailSchema, phoneSchema, notesSchema, createUpdateSchema } from
'@repo/core'


// 1. TypeScript interface (what a record looks like)

export interface ClinicPatient {

id: string // Auto-generated UUID

tenantId: string // Which client owns this — REQUIRED

patientCode: string

firstName: string

lastName: string

// ... your fields ...

createdAt: number // Auto-set timestamp

updatedAt: number // Auto-updated timestamp

deletedAt: number | null // null = not deleted (soft delete)

version: number // For sync conflict detection

createdBy: string // Who created it

updatedBy: string // Who last updated it

}


// 2. Zod validation (enforced on every create/update)

export const CreateClinicPatientSchema = z.object({

tenantId: z.string().min(1),

patientCode: z.string().min(1).max(20),

firstName: z.string().min(1).max(100),

lastName: z.string().min(1).max(100),

phone: phoneSchema.optional(),

email: emailSchema.optional(),
```

```
  // ... your fields ...

  })


  export const UpdateClinicPatientSchema = createUpdateSchema({

  firstName: z.string().min(1).max(100).optional(),

  // ... updatable fields ...

  })
```

- ■■ ***Always include*** `id`, `tenantId`, `createdAt`, `updatedAt`, `deletedAt`, `version`, `createdBy`, `updatedBy` *in every interface. The framework requires these.*

`clinic.entity.ts` — *Tell the framework about your entity*

```
import { EntityRegistry } from '@repo/core'

export const ClinicPatientEntity = {
name: 'clinic_patients', // Must match DB table name EXACTLY
ui: {
label: 'Patient',
labelPlural: 'Patients',
icon: 'Users', // Lucide icon name
routePath: 'clinic/patients',
color: 'blue',
showInNav: true,
navOrder: 10, // Lower = higher in sidebar
navGroup: 'Clinic', // Sidebar section header
},
sync: { enabled: true, conflictStrategy: 'lww', priority: 'normal' },
audit: { enabled: true },
rbac: { enabled: true, permissionPrefix: 'clinic_patient' },
hooks: {},
pagination: 'offset',
tenant: { enabled: true, field: 'tenantId' },
softDelete: { enabled: true, field: 'deletedAt' },
}

// This ONE LINE makes the framework aware of your entity
EntityRegistry.register(ClinicPatientEntity)
```

`clinic.service.ts` — *Pure business logic (no DB calls here)*

```
export class ClinicPatientService {

static computeAge(dateOfBirth: string): number {

const dob = new Date(dateOfBirth)

const today = new Date()

let age = today.getFullYear() - dob.getFullYear()

const m = today.getMonth() - dob.getMonth()

if (m < 0 || (m === 0 && today.getDate() < dob.getDate())) age--

return age

}


static generatePatientCode(sequenceNumber: number): string {

const now = new Date()

const ym = `${now.getFullYear()}${String(now.getMonth()+1).padStart(2,'0')}`

return `PT-${ym}-${String(sequenceNumber).padStart(4,'0')}`

}


static prepareForCreate(input: Record<string, unknown>) {

return {

...input,

status: input.status ?? 'active',

fullName: `${input.firstName} ${input.lastName}`.trim(),

}

}

}
```

`clinic.policies.ts` ***— Role-Based Access Control***

```
export const ClinicPatientPolicies = [

{ effect: 'allow', action: '*',

conditions: (ctx) => ctx.roles.includes('admin') ||
ctx.roles.includes('clinic_admin'),

priority: 100 },

{ effect: 'allow', action: 'read',

conditions: (ctx) => ctx.roles.some(r => ['doctor', 'nurse',
'receptionist'].includes(r)),

priority: 80 },

{ effect: 'deny', action: 'delete', priority: 50 }, // Prevent hard deletes

]
```

`index.ts` ***— Barrel export (public API)***

```
// Self-register all entities when package is imported

export { ClinicPatientEntity, ClinicDoctorEntity } from './clinic.entity'


// Export types and schemas

export type { ClinicPatient } from './clinic.schema'

export { CreateClinicPatientSchema, UpdateClinicPatientSchema } from
'./clinic.schema'

export { ClinicPatientService } from './clinic.service'

export { ClinicPatientPolicies } from './clinic.policies'
```

STEP 3: Add Database Tables

Open: `packages/db-adapter-dexie/src/index.ts`

Add a **new** database version (NEVER edit existing versions):

```
// Find the last this.version(N) block, then add N+1 after it:
```

```
this.version(4).stores({ // <- Use the NEXT sequential number

clinic_patients: 'id, tenantId, patientCode, firstName, lastName, status, createdAt,
updatedAt, deletedAt, [tenantId+deletedAt], [tenantId+status]',

clinic_doctors: 'id, tenantId, doctorCode, status, createdAt, updatedAt, deletedAt,
[tenantId+status]',

clinic_appointments: 'id, tenantId, patientId, doctorId, appointmentDate, status,
createdAt, updatedAt, deletedAt, [patientId+appointmentDate],
[doctorId+appointmentDate]',

})
```

Index syntax guide:

- Single field: `fieldName` — basic index
- Compound: `[field1+field2]` — combined index for multi-field queries
- Always index: `id, tenantId, createdAt, updatedAt, deletedAt`
- Add compound: `[tenantId+deletedAt]` for "all active records for this tenant"
- Add compound: `[tenantId+status]` for "filter by status within tenant"

> • ■■ ***Why new version?*** *IndexedDB requires a version bump to add tables. Dexie auto-migrates — existing data is NEVER lost. Just always increment.*

STEP 4: Wire Into `apps/web/src/lib/db.ts`

Three things to add:

```
// A) Type imports (top of file, with other type imports)

import type { ClinicPatient, ClinicDoctor, ClinicAppointment } from
'@repo/entity-clinic'


// B) Self-registration import (with other entity package imports)

import '@repo/entity-clinic'

// This one import automatically:

// - Registers all clinic entities with EntityRegistry

// - Adds "Clinic" group to the sidebar nav
```

```
// - Enables sync, audit, RBAC for all clinic entities


// C) Repository instances (with other repo declarations)

export const clinicPatientRepo: Repository<ClinicPatient> =
createRepo<ClinicPatient>('clinic_patients')

export const clinicDoctorRepo: Repository<ClinicDoctor> =
createRepo<ClinicDoctor>('clinic_doctors')

// One createRepo() call per database table
```

STEP 5: Add Package Dependency

In `apps/web/package.json`:

```
{

"dependencies": {

"@repo/entity-clinic": "workspace:*",

"@repo/entity-customer": "workspace:*",

// ... other deps

}

}
```

Then run:

```
pnpm install
```

STEP 6: Create UI Pages

Create route files for each entity view:

```
apps/web/src/routes/

■■■ clinic/

■■■ patients/
```

```
■ ■■■ index.tsx <- List: table of all patients
■ ■■■ new.tsx <- Create: registration form
■ ■■■ $id.tsx <- Detail/Edit: single patient view
■■■ appointments/
■ ■■■ index.tsx
■■■ billing/
■■■ index.tsx
```

Minimum working list page:

```
import { useEffect, useState, useCallback } from 'react'
import { useNavigate } from '@tanstack/react-router'
import { Card, CardHeader, Button } from '@repo/ui-core'
import { clinicPatientRepo } from '../../lib/db'

export function ClinicPatientsPage() {
const navigate = useNavigate()
const [patients, setPatients] = useState([])
const [loading, setLoading] = useState(true)

const load = useCallback(async () => {
setLoading(true)
const result = await clinicPatientRepo.findMany({
page: 1,
pageSize: 20,
sort: [{ field: 'lastName', direction: 'asc' }],
})
if ('items' in result) setPatients(result.items)
setLoading(false)
}, [])
```

```
useEffect(() => { load() }, [load])

return (
<div className="p-6">
<Card>
<CardHeader
title="Patients"
description={`${patients.length} patients`}
action={<Button onClick={() => navigate({ to: '/clinic/patients/new' })}>Add
Patient</Button>}
/>
{/* Render table rows */}
</Card>
</div>
)
}
```

STEP 7: Register Routes

In `apps/web/src/router.tsx`:

```
// A) Import your page components (top of file)
import { ClinicPatientsPage } from './routes/clinic/patients/index'
import { ClinicNewPatientPage } from './routes/clinic/patients/new'
import { ClinicPatientDetailPage } from './routes/clinic/patients/$id'

// B) Add to routeTree.addChildren([...]) array
routeTree.addChildren([
// ... existing routes ...
```

```
// Clinic Management System

createRoute({ getParentRoute: () => rootRoute, path: 'clinic/patients', component:
ClinicPatientsPage }),

createRoute({ getParentRoute: () => rootRoute, path: 'clinic/patients/new',
component: ClinicNewPatientPage }),

createRoute({ getParentRoute: () => rootRoute, path: 'clinic/patients/$id',
component: ClinicPatientDetailPage }),

createRoute({ getParentRoute: () => rootRoute, path: 'clinic/appointments',
component: ClinicAppointmentsPage }),

createRoute({ getParentRoute: () => rootRoute, path: 'clinic/billing', component:
ClinicBillingPage }),

])
```

STEP 8: Add Navigation Group (If New Group)

In `apps/web/src/hooks/useDynamicNav.ts`:

```
const ROUTE_TO_GROUP: Record<string, string> = {

// ... existing entries ...

'/clinic': 'Clinic', // <- maps URL prefix to sidebar group name

}
```

> ***Why this works:*** *The `navGroup: 'Clinic'` in your entity definition sets which group it appears under. The `useDynamicNav()` hook reads from EntityRegistry and groups items automatically. The ROUTE_TO_GROUP is just a fallback for non-entity routes.*

STEP 9: Add Feature Flag

In `packages/feature-flags/src/index.ts`:

```
featureFlags.defineMany([

// ... existing flags ...

{
```

```
  key: 'module.clinic',
  description: 'Enable Clinic Management System for medical/healthcare clients',
  enabled: true,
  // Optional: restrict to specific tenant IDs:
  // rules: [{ target: 'tenant', values: ['medilife-clinic-manila'] }],
  },
  ])
```

STEP 10: Install and Test

```
  pnpm install # Link the new workspace package
  pnpm dev # Start dev server

  # Verify:
  # 1. Left sidebar shows "Clinic" section with Patients, Appointments, etc.
  # 2. Clicking "Patients" goes to /clinic/patients
  # 3. Creating a patient saves offline (IndexedDB)
  # 4. Refreshing the page — data persists (offline-first!)
  # 5. No console errors
```

---

5. Real Example: Clinic Management System

This is exactly what was built in this codebase:

File Created	Purpose
`packages/entity-clinic/package.json`	Package config
`packages/entity-clinic/src/clinic.schema.ts`	5 entity schemas (patient, doctor, appointment, record, billing)

File Created	Purpose
packages/entity-clinic/src/clinic.entity.ts	5 EntityDefinitions + EntityRegistry.register()
packages/entity-clinic/src/clinic.service.ts	Age calculation, code generation, billing math (PHP)
packages/entity-clinic/src/clinic.policies.ts	RBAC for admin, clinic_admin, doctor, nurse, receptionist, cashier
packages/entity-clinic/src/index.ts	Barrel export
packages/db-adapter-dexie/src/index.ts	v3 schema: 5 clinic tables with compound indexes
apps/web/src/lib/db.ts	5 clinic repos, type imports, package import
apps/web/package.json	@repo/entity-clinic: workspace:* dependency
apps/web/src/routes/clinic/patients/index.tsx	Patient list with search/filter/pagination
apps/web/src/routes/clinic/patients/new.tsx	Patient registration form (5 sections)
apps/web/src/routes/clinic/patients/$id.tsx	Patient detail view
apps/web/src/routes/clinic/appointments/index.tsx	Today's appointment schedule with date picker
apps/web/src/routes/clinic/billing/index.tsx	Billing list with PHP currency (■)
apps/web/src/router.tsx	All clinic routes registered
apps/web/src/hooks/useDynamicNav.ts	/clinic → 'Clinic' group mapping
packages/feature-flags/src/index.ts	module.clinic feature flag

Entities created:

- `ClinicPatient` — Demographics, blood type, allergies, emergency contact, address
- `ClinicDoctor` — Profile, specialization, PRC license, consultation fee (PHP)
- `ClinicAppointment` — Scheduling, chief complaint, status tracking
- `ClinicConsultationRecord` — SOAP notes + vitals (BP, pulse, temp, weight, SpO2, ICD code)
- `ClinicBilling` — Fee breakdown, PhilHealth/HMO support, balance tracking in PHP

---

6. Deploying to a New Client

When a new client signs up:

Step A: Assign a Tenant ID

```
Cooperative: "coop-batangas-2024"

Clinic: "medilife-clinic-manila"

Water station: "crystal-water-bulacan"
```

This is permanent. Never change it after data is created.

Step B: Create Tenant Metadata

Seed their config via the admin panel or a seed script:

```
await metadataStore.set('medilife-clinic-manila', {

ui: {

theme: { primaryColor: '#2563eb', logo: 'https://cdn.example.com/logo.png' },

customFields: {

clinic_patients: [

{ name: 'philhealthNumber', label: 'PhilHealth No.', type: 'text', required: false }

]

}
```

```
},

clinic: {

defaultConsultationFee: 500,

appointmentSlotMinutes: 30,

}

})
```

Step C: Create User Accounts

```
receptionist@medilife.com → roles: ['receptionist'], tenantId:
'medilife-clinic-manila'

dr.santos@medilife.com → roles: ['doctor'], tenantId: 'medilife-clinic-manila'

admin@medilife.com → roles: ['clinic_admin'], tenantId: 'medilife-clinic-manila'
```

Step D: Enable/Disable Modules via Feature Flags

```
// Feature flag rules restrict to specific tenants

rules: [{ target: 'tenant', values: ['medilife-clinic-manila'] }]
```

Step E: Share the URL

This is a PWA. No installation needed. Client opens:

```
https://yourapp.netlify.app
```

They log in → their tenantId is set → they see only their data.

---

7. Maintaining Client Systems

Adding a New Field — Option A: Custom Field (No Deployment)

```
// Admin sets via metadata — zero code change needed

await metadataStore.setField('medilife-clinic-manila',

'ui.customFields.clinic_patients',

[{ name: 'insuranceNumber', label: 'Insurance No.', type: 'text', required: false }]

)
```

The `GenericForm` component reads custom fields from metadata and renders them automatically.

Adding a New Field — Option B: Permanent Schema Field (Code Change)

1. Add to TypeScript interface in `clinic.schema.ts`
2. Add to Zod schema
3. Add a new Dexie version:

```
this.version(5).stores({

clinic_patients: '...existing..., insuranceNumber', // Re-declare with new index

})
```

4. Deploy. Dexie auto-migrates. Old records have `undefined` for the new field — add defaults in the service.

Updating Business Rules

```
// No code change needed — update per-tenant metadata

await metadataStore.setField('medilife-clinic-manila',
'clinic.defaultConsultationFee', 750)
```

Disabling a Module for One Client

```
// In feature flags — add tenant exclusion rule

rules: [{ target: 'exclude_tenant', values: ['old-client-id'] }]
```

8. Managing Multiple Clients in One Database

Local (Browser) — Fully Isolated

Each browser = its own IndexedDB. Client A's browser physically cannot contain Client B's data.

Server (Supabase) — Logically Isolated

All clients share one database. Isolation is enforced by:

1. **Tenant Middleware** — Every query adds `WHERE tenantId = ?` automatically
2. **Row-Level Security** — Database-level Supabase policies
3. **Audit Trail** — Every change logged with `tenantId`, `userId`, `timestamp`

Debug Commands

```
// Browser console (for dev/support)

window.__DB__.clinicPatientRepo.findMany({ page: 1, pageSize: 100 })

window.__METADATA__.get('medilife-clinic-manila')
```

```
-- Supabase dashboard

SELECT * FROM clinic_patients WHERE "tenantId" = 'medilife-clinic-manila';

SELECT "tenantId", COUNT(*) FROM clinic_patients GROUP BY "tenantId";
```

---

9. Tenant Metadata — Per-Client Customization Without Code

The metadata store is a JSON blob per tenant. Change it via the admin panel — no deployment needed.

What Can Go in Metadata

```
{
"ui": {
"theme": { "primaryColor": "#2563eb", "logo": "https://..." },
"customFields": {
"clinic_patients": [
{ "name": "philhealthNumber", "label": "PhilHealth No.", "type": "text" }
]
}
},
"clinic": {
"defaultConsultationFee": 500,
"appointmentSlotMinutes": 30,
"allowedPaymentMethods": ["cash", "gcash", "philhealth"]
},
"approvalWorkflows": {
"billing": {
"steps": [
{ "role": "cashier", "action": "encode" },
{ "role": "clinic_admin", "action": "approve", "minAmount": 5000 }
]
}
}
}
```

Reading Metadata in Code

```
import { metadataResolver } from '../../lib/db'

const customFields = await metadataResolver.getCustomFields(tenantId,
'clinic_patients')

const uiConfig = await metadataResolver.getUIConfig(tenantId)

const workflow = await metadataResolver.getApprovalWorkflow(tenantId, 'billing')
```

Adding New Metadata Domains for Your Module

In `packages/multi-tenant/src/metadata-resolver.ts`:

```
async getClinicConfig(tenantId: string) {

const metadata = await this.getMetadata(tenantId)

const clinic = metadata.clinic ?? {}

return {

defaultConsultationFee: clinic.defaultConsultationFee ?? 500,

appointmentSlotMinutes: clinic.appointmentSlotMinutes ?? 30,

}

}
```

---

10. Feature Flags — Turning Modules On/Off Per Client

Flag Key	Default	Controls
`sync.enabled`	ON	Background sync to server
`sync.realtime`	OFF	WebSocket real-time updates
`audit.enabled`	ON	Audit trail logging
`export.csv`	ON	CSV export

Flag Key	Default	Controls
`export.pdf`	OFF	PDF export (enterprise)
`multi-tenant`	ON	Multi-tenancy features
`module.clinic`	ON	Clinic Management System

Defining Flags

```
featureFlags.defineMany([{

key: 'module.your-module',

description: 'Human-readable description',

enabled: true,

// Option A: Specific tenants only

rules: [{ target: 'tenant', values: ['tenant-id-1'] }],

// Option B: Dev only

rules: [{ target: 'environment', environments: ['development'] }],

// Option C: Gradual rollout

rules: [{ target: 'percentage', percentage: 50 }],

}])
```

Checking Flags in Components

```
import { useFeatureFlag } from '../context/FeatureFlagContext'


export function MyComponent() {

const isEnabled = useFeatureFlag('module.clinic')

if (!isEnabled) return null

return <ClinicDashboard />

}
```

11. Troubleshooting Common Issues

Problem	Cause	Fix
Module not in sidebar	Missing `import '@repo/entity-clinic'` in `lib/db.ts`	Add the import
Module not in sidebar	`showInNav: false` in entity definition	Set to `true`
Module not in sidebar	navGroup not in ROUTE_TO_GROUP	Add to `useDynamicNav.ts`
"Entity not registered" error	Package not imported	Add `import '@repo/entity-clinic'` to `lib/db.ts`
Data disappears on refresh	Table name mismatch	Verify `createRepo('table_name')` matches Dexie schema
TypeScript import errors	Package not in `package.json`	Add dep + `pnpm install`
Dexie VersionError	Duplicate version number	Use next sequential integer for version
Tenant data leaked to other	Middleware missing	Verify `createRepo()` goes through `createDexieRepository`

---

12. Quick Reference Cheatsheet

File Map — What to Edit for What Task

Task	File to Edit
Define data shape + validation	`packages/entity-[name]/src/[name].schema.ts`
Register entity with framework	`packages/entity-[name]/src/[name].entity.ts`
Add business logic	`packages/entity-[name]/src/[name].service.ts`
Define access control (roles)	`packages/entity-[name]/src/[name].policies.ts`
Export the module	`packages/entity-[name]/src/index.ts`
Add database tables	`packages/db-adapter-dexie/src/index.ts`
Wire repositories + import	`apps/web/src/lib/db.ts`
Add package dependency	`apps/web/package.json`
Add URL routes	`apps/web/src/router.tsx`
Create UI pages	`apps/web/src/routes/[module]/`
Add nav sidebar group	`apps/web/src/hooks/useDynamicNav.ts`
Add feature toggle	`packages/feature-flags/src/index.ts`
Add per-client config	`packages/multi-tenant/src/metadata-resolver.ts`

Commands

```
pnpm dev # Start dev server

pnpm codegen entity [Name] # Scaffold new entity package

pnpm install # Install after adding new packages
```

```
pnpm -r typecheck # Type-check all packages

pnpm -r test # Run all tests

pnpm build # Production build
```

Repository API Reference

```
// Create a record

const record = await myRepo.create({ tenantId: 'client-id', ...data })


// Find one by ID

const record = await myRepo.findById('uuid-here')


// Find many (with filter, search, pagination)

const result = await myRepo.findMany({

page: 1,

pageSize: 20,

filter: [{ field: 'status', operator: 'eq', value: 'active' }],

search: 'search term',

sort: [{ field: 'createdAt', direction: 'desc' }],

})

// result.items = array

// result.total = total count


// Update

const updated = await myRepo.update('uuid', { ...changes, version: currentVersion })


// Soft delete

await myRepo.delete('uuid')


// Count

const count = await myRepo.count({ filter: [...] })
```

Entity Definition Template

```
const MyEntity: EntityDefinition<MyType> = {

name: 'my_table_name', // Matches Dexie table name EXACTLY

ui: {

label: 'Singular',

labelPlural: 'Plural',

icon: 'Layers', // Lucide icon name

routePath: 'module/path', // No leading slash

color: 'blue', // blue|green|red|yellow|purple|gray

showInNav: true,

navOrder: 50, // Lower = higher in sidebar

navGroup: 'Module Name', // Sidebar section header

},

sync: { enabled: true, conflictStrategy: 'lww', priority: 'normal' },

audit: { enabled: true },

rbac: { enabled: true, permissionPrefix: 'my_entity' },

hooks: {},

pagination: 'offset',

tenant: { enabled: true, field: 'tenantId' },

softDelete: { enabled: true, field: 'deletedAt' },

}

EntityRegistry.register(MyEntity) // <-- Self-register!
```

■ The Complete Flow in One Picture

```
1. Plan entities (what tables do you need?)
```

```
  |
  v
  2. Create packages/entity-[name]/src/
  (schema + entity + service + policies + index)
  |
  v
  3. Add DB tables in db-adapter-dexie (new version number!)
  |
  v
  4. Import + wire repos in apps/web/src/lib/db.ts
  |
  v
  5. Add dependency in apps/web/package.json + pnpm install
  |
  v
  6. Create UI pages in apps/web/src/routes/[module]/
  |
  v
  7. Register routes in apps/web/src/router.tsx
  |
  v
  8. Add nav group to useDynamicNav.ts (if new section)
  |
  v
  9. Add feature flag in feature-flags/src/index.ts
  |
  v
```

```
10. pnpm dev -> Test it!
```

- ***■ Once you complete steps 1–10, navigation, sync, audit trail, RBAC, offline support, and tenant isolation are all automatic.***

Last updated: 2026-07-13 | Offline-First Business App Starter Kit